ProxSuite: Removing the veg library

Removing `veg`

ProxSuite bundled `proxsuite::linalg::veg`, a metaprogramming library that emulated concepts, tuples, tags, function objects and allocators with C++11 preprocessor machinery. ProxSuite requires C++17, so all of it has a standard spelling. The library has been removed: 41 headers, about 10 300 lines.

This page covers the rationale, the changes, the breaking changes, and the migration path.

---

1. Why

`veg` was a vendored fork with no upstream. Every bug fix, compiler workaround and new warning class landed on ProxSuite's maintainers, for code whose only purpose was to spell C++17 in C++11.

Costs:

- **Unfamiliar API.** `VEG_TEMPLATE`, `VEG_NIEBLOID`, `VEG_BIND`, `Tag<T>`, `unsafe`, `from_raw_parts` are project-specific. None of them is documented outside this repository.
- **Poor diagnostics.** A type error inside a `VEG_TEMPLATE` produced a macro expansion trace instead of a candidate list.
- **Macro pollution.** `veg` and ProxSuite defined unprefixed macros in installed headers (`LDLT_TEMP_VEC`, `LDLT_ID`, `PROX_QP_ALL_OF`, …) with no `#undef`, so they leaked into every translation unit including ProxSuite. Some used a leading double underscore, which is reserved to the implementation ([lex.name]/3).
- **Build time.** ~10 000 lines of header-only template and preprocessor code parsed by every translation unit.

One component was worth keeping: the bump allocator used for solver scratch space. It is reimplemented in 467 lines (`proxsuite/linalg/dynstack.hpp` and `slice.hpp`).

---

2. How the update was done

The `veg` headers were mutually dependent, so the port could not be incremental. The removal is one commit. The surrounding work was split to keep each step reviewable.

Step	Rationale
Replace `PROXSUITE_DEDUCE_RET` with return type deduction	Five call sites, independent of the rest
Add a standalone `DynStack` / `Slice`	The replacement must exist before the original is deleted
Drop `veg` and port every call site	One commit; the headers cannot be split
Delete dead pre-C++17 fallbacks	`PROXSUITE_WITH_CPP_14/17` and `PROXSUITE_MAYBE_UNUSED` are unreachable under C++17
Replace `PROX_QP_ALL_OF` / `PROXSUITE_CHECK_SIZE`	Without `veg` these hide only a function call; `PROXSUITE_CHECK_SIZE` hid a `return false` at its call sites
Prefix or `#undef` the leaking macros	Separate commit, so the rename is easy to review and revert
Convert include guards to `#pragma once`	70 headers, mechanical
Restore the explicit-instantiation macros	22 `extern template` declarations written by hand cost more than one self-contained macro

Validation. The full C++ and Python test suites pass unchanged, including with assertions enabled. No test was modified except to follow the renamed API.

---

3. Changes

3.1 Concepts

`VEG_DEF_CONCEPT` generated a class template plus a macro to query it. C++17 has `constexpr` variable templates.

```
// before
VEG_DEF_CONCEPT(typename T, rvalue_ref, std::is_rvalue_reference<T>::value);
VEG_DEF_CONCEPT((typename Mat, typename T),
                has_data_expr,
                LDLT_CONCEPT(detected<detail::DataExpr, Mat, T>));

// after
template<typename T>
inline constexpr bool rvalue_ref = std::is_rvalue_reference<T>::value;

template<typename Mat, typename T>
inline constexpr bool has_data_expr =
  concepts::detected<detail::DataExpr, Mat, T>;
```

Constrained templates went from `VEG_TEMPLATE(...) requires(...)` to `std::enable_if_t`. There are six such sites, all constructors:

```
template<typename Vec,
         typename = std::enable_if_t<concepts::eigen_vector_view<Vec, T>>>
VectorView(FromEigen, Vec const& vec) noexcept;
```

3.2 Function objects (“niebloids”)

`VEG_NIEBLOID` wrapped a struct with a templated `operator()` in a global object. These are function templates now. The call syntax is identical, so most call sites needed no edit.

```
// before
namespace nb {
struct zero_extend
{
  template<typename I>
  auto operator()(I a) const noexcept -> usize
  {
    return usize(typename std::make_unsigned<I>::type(a));
  }
};
} // namespace nb
VEG_NIEBLOID(zero_extend);

// after
template<typename I>
auto
zero_extend(I a) noexcept -> usize
{
  return usize(typename std::make_unsigned<I>::type(a));
}
```

Storing one in a variable no longer works:

```
auto zx = util::zero_extend;                          // before
auto zx = [](auto i) { return util::zero_extend(i); };  // after
```

3.3 Tuples and destructuring

```
// before
VEG_BIND(auto,
         (_, new_current_col, computed_difference),
         sparse::merge_second_col_into_first(/* ... */));

// after
auto [merged_values, new_current_col, computed_difference] =
  sparse::merge_second_col_into_first(/* ... */);
```

3.4 Type tags became template arguments

`veg::Tag<T>` was a value parameter carrying a type, because C++11 deduction was easier to steer that way. C++17 does not need it.

```
// before
auto req = proxsuite::linalg::dense::factorize_req(veg::Tag<T>{}, n);
auto tmp = stack.make_new_for_overwrite(veg::Tag<T>{}, n);

// after
auto req = proxsuite::linalg::dense::factorize_req<T>(n);
auto tmp = stack.make_new_for_overwrite<T>(n);
```

Internal signatures simplify too: `refactorize()` lost a trailing `veg::Tag<T>& xtag` parameter that only named a type.

3.5 Owning buffers became `std::vector`

`veg::Vec<T>` was an owning container with a custom allocator, not a view. The view types were `veg::Slice` / `SliceMut`, which map to `proxsuite::linalg::Slice` / `SliceMut`. The allocator never requested more alignment than `operator new` guarantees, so `std::vector`'s default allocator is behaviourally identical.

```
// before
proxsuite::linalg::veg::Vec<I> etree;
proxsuite::linalg::veg::Vec<I> perm;
proxsuite::linalg::veg::Vec<proxsuite::linalg::veg::mem::byte> storage;

// after
std::vector<I> etree;
std::vector<I> perm;
std::vector<unsigned char> storage;
```

```
// before
data.kkt_col_ptrs.resize_for_overwrite(n_tot + 1);
I* kktp = data.kkt_col_ptrs.ptr_mut();

// after
data.kkt_col_ptrs.resize(usize(n_tot + 1));
I* kktp = data.kkt_col_ptrs.data();
```

No copies were introduced. Both types own their storage and both move. Every `std::vector` in the installed headers is either a data member or a reference parameter; none is passed or returned by value.

One semantic difference: `veg::Vec`'s copy constructor was `explicit` (`VEG_EXPLICIT_COPY`), so implicit copies were a compile error and had to be written as `Vec<T> b(a);`. `std::vector` copies implicitly. Copy *assignment* was already implicit in both. The risk is therefore future accidental copies, not existing ones: an implicit copy that used to fail to compile now succeeds silently. Pass these members by reference.

3.6 Assertions

```
VEG_ASSERT_ALL_OF(a.nrows() == out_r.dim,     // before
                  a.ncols() == in_r.dim);

assert(a.nrows() == out_r.dim);               // after
assert(a.ncols() == in_r.dim);
```

3.7 The dynamic stack

`StackReq` keeps its meaning: `a & b` is "both allocations live at the same time", `a | b` is "either one, whichever is larger", `alloc_req()` is the byte count to allocate. The implementation changed: a borrowed array is a single move-only RAII type instead of a CRTP base plus a destructor mixin plus a rollback guard, and the LIFO release order is asserted in debug builds rather than assumed.

```
// before
VEG_MAKE_STACK(stack, req);
auto work = stack.make_new_for_overwrite(veg::Tag<T>{}, n);

// after
std::vector<unsigned char> stack_storage(proxsuite::usize(req.alloc_req()));
proxsuite::linalg::dynstack::DynStackMut stack{
  stack_storage.data(), proxsuite::isize(stack_storage.size())
};
auto work = stack.make_new_for_overwrite<T>(n);
```

3.8 Macro hygiene

A macro defined by a public header without `#undef` applies to every including translation unit. The SIMD helpers used only inside `dense/core.hpp` are now `#undef`'d after use; the rest took the project prefix.

```
LDLT_TEMP_VEC(T, tmp, n, stack);             // before
PROXSUITE_LDLT_TEMP_VEC(T, tmp, n, stack);   // after
```

`PROXSUITE_EXPLICIT_TPL_DECL` / `_DEF` keep the `LDLT_EXPLICIT_TPL_*` call syntax (parameter count, then function name), backed by one macro per arity plus a small `FnInfo` trait instead of a preprocessor loop:

```
PROXSUITE_EXPLICIT_TPL_DECL(2, llt_compute<Mat<f32, colmajor>>);
```

3.9 Latent bug fixed

The old cereal `load` for `veg::Vec<bool>` called `reserve(len)` and then wrote through `operator[]`, past `size()`. Round-tripping a non-empty `Results::active_constraints` was undefined behaviour. Cereal's own `std::vector` support replaces it.

---

4. Impact

You are	Impact
A Python user	None. The bindings expose the same names and types.
A C++ user of `proxqp::dense::QP` / `proxqp::sparse::QP` only	None, unless you read the workspace or model buffers directly.
A C++ user who names `proxsuite::linalg::veg::...`	The namespace is gone. See §5.
A C++ user of the `linalg` backends (`Ldlt`, sparse factorization)	The `*_req` signatures changed. See §5.3.
A Python user unpickling `Results` / `QP` from an older ProxSuite	The pickle will not load. See §5.8.
Anyone reading JSON or XML archives from an older ProxSuite	Not compatible. Binary archives are unaffected on 64-bit.

Unchanged: the solver API (`QP::init`, `QP::solve`, `QP::update`, `Settings`, `Results`, `Model`, the free `solve` functions), numerical behaviour, and the alignment of the dense factorization storage.

---

5. Migration guide

5.1 Headers

Every `proxsuite/linalg/veg/**` header is gone. Two replace them:

```
#include <proxsuite/linalg/dynstack.hpp>   // StackReq, DynStackMut, DynStackArray
#include <proxsuite/linalg/slice.hpp>      // Slice, SliceMut
```

The rest maps onto the standard library: `<vector>`, `<tuple>`, `<type_traits>`, `<optional>`.

`PROXSUITE_THROW_PRETTY`, `PROXSUITE_CHECK_ARGUMENT_SIZE` and `PROXSUITE_PRETTY_FUNCTION` were defined in a `veg` header and often picked up transitively. They now live in `<proxsuite/helpers/common.hpp>`.

Headers use `#pragma once`, so the `PROXSUITE_*_HPP` include-guard macros are no longer defined. Do not test for them.

5.2 Namespaces and types

Was	Is now
`veg::isize`, `veg::usize`	`proxsuite::isize`, `proxsuite::usize`
`veg::i32`, `veg::u64`, …	`std::int32_t`, `std::uint64_t`, …
`veg::uncvref_t<T>`	`proxsuite::remove_cvref_t<T>`
`veg::DoNotDeduce<T>`	`proxsuite::DoNotDeduce<T>`
`veg::Slice<T>`, `veg::SliceMut<T>`	`proxsuite::linalg::Slice<T>`, `SliceMut<T>`
`veg::dynstack::StackReq` etc.	`proxsuite::linalg::dynstack::StackReq` etc.
`veg::Vec<T>`	`std::vector<T>`
`veg::Tuple<...>`, `veg::tuplify(...)`	`std::tuple<...>`, `std::make_tuple(...)`
`veg::meta::if_t`	`std::conditional_t`
`veg::meta::bool_constant`	`std::bool_constant`
`veg::meta::constant<T, V>`	`std::integral_constant<T, V>`
`veg::meta::unptr_t`	`std::remove_pointer_t`

5.3 `*_req` functions

Every `*_req` function traded its `veg::Tag<T>` parameter for an explicit template argument (§3.4). This applies to `transpose_req`, `transpose_symbolic_req`, `etree_req`, `postorder_req`, `column_counts_req`, `amd_req`, `symmetric_permute_req`, `symmetric_permute_symbolic_req`, `factorize_symbolic_req`, `factorize_numeric_req`, `merge_second_col_into_first_req`, `rank1_update_req`, `factorize_unblocked_req`, `factorize_blocked_req`, `factorize_recursive_req`, `factorize_req`, `ldlt_insert_rows_and_cols_req`, `ldlt_delete_rows_and_cols_req`, `temp_vec_req`, `temp_mat_req`, and the row-modification requests.

`Preconditioner::scale_qp_in_place_req` drops the tag entirely; its scalar type comes from the class:

```
        P::scale_qp_in_place_req(n, n_eq, n_in);   // was P::scale_qp_in_place_req(veg::Tag<T>{}, ...)
```

5.4 Public members that changed type

- `proxqp::Results<T>::active_constraints` -> `std::vector<bool>`
- `proxqp::dense::Workspace<T>::ldl_stack`, `::alphas`
- `proxqp::sparse::Model<T, I>`: the six `kkt_*` arrays
- `proxqp::sparse::Workspace<T, I>`: `storage`, `kkt_nnz_counts`, and `ldl.{etree, perm, perm_inv, col_ptrs, nnz_counts, row_indices, values}`
- `linalg::dense::Ldlt<T>`: its internal storage (private, but the ABI changed)

Accessor mapping:

`veg::Vec`	`std::vector`
`v.len()`	`isize(v.size())` — note the signedness
`v.ptr()`, `v.ptr_mut()`	`v.data()`
`v.push(x)`	`v.push_back(x)`
`v.resize_for_overwrite(n)`	`v.resize(usize(n))`
`v.reserve_exact(n)`	`v.reserve(usize(n))`
`v.pop_mid(i)`	`v.erase(v.begin() + i)`
`v.push_mid(x, i)`	`v.insert(v.begin() + i, x)`
`v.as_mut()`	`SliceMut<T>{ v.data(), isize(v.size()) }`
`v.as_ref()`	`Slice<T>{ v.data(), isize(v.size()) }`

`resize_for_overwrite` left new elements uninitialized; `resize` value-initializes them. The extra zero-fill is `O(n)` on buffers

whose users are `O(n²)` or worse, and only occurs on setup paths.

One member did not become a `std::vector`: `proxqp::sparse::Workspace<T, I>::active_inequalities` is a `VecBool`, the Eigen boolean vector the dense workspace already used. The solver takes a contiguous `SliceMut<bool>` of it, which the bit-packed `std::vector<bool>` cannot provide.

5.5 Slices and the stack

```
Slice<T>{ veg::unsafe, veg::from_raw_parts, ptr, len }  // before
proxsuite::linalg::Slice<T>{ ptr, len }                 // after
```

The interface is `ptr()`, `ptr_mut()`, `len()`, `is_empty()`, `operator[]`, `begin()`, `end()`, and `as_const()` on the mutable one. `SliceMut<T>` converts implicitly to `Slice<T>`; bounds are asserted in debug builds. `split_at`, `get_unchecked`, `as_bytes` and `veg::Array` were unused and were not carried over.

`make_new` / `make_new_for_overwrite` still take an optional trailing alignment. `make_alloc` is gone; nothing used it. The returned `DynStackArray<T>` is move-only and releases on scope exit, in reverse order of acquisition. `PROX_QP_ALL_OF` / `PROX_QP_ANY_OF` are gone; call `StackReq::and_({...})` and `StackReq::or_({...})`.

5.6 Sparse matrix views

`MatRef`, `MatMut`, `SymbolicMatRef` and `SymbolicMatMut` no longer use a CRTP interface; they derive from a small shared base. Their public interface is unchanged apart from:

- the `Unsafe` tag is dropped from `col_start_unchecked(j)` / `col_end_unchecked(j)`, and from `proxqp::sparse::detail::top_rows_unchecked` / `top_rows_mut_unchecked`;
- `MatMut::symbolic_mut()` is removed; nothing used it;
- `col_ptrs_mut()`, `nnz_per_col_mut()`, `row_indices_mut()` and `values_mut()` are `const`-qualified, so they can be called on a const view. The view is const, the data it points at is not.

Their layout changed, so this is an ABI break.

5.7 Macros

Removed:

Macro	Replacement
all `VEG_*`	the standard C++17 construct it emulated
`VEG_BIND(auto, (a, b), expr)`	`auto [a, b] = expr;`
`LDLT_CONCEPT(x)`, `LDLT_CHECK_CONCEPT(x)`	`concepts::x`, `static_assert(concepts::x)`
`PROXSUITE_DEDUCE_RET`	C++14 return type deduction
`PROXSUITE_WITH_CPP_14`, `PROXSUITE_WITH_CPP_17`	nothing; C++17 is required
`PROXSUITE_MAYBE_UNUSED`	`[[maybe_unused]]`
`PROX_QP_ALL_OF`, `PROX_QP_ANY_OF`	`StackReq::and_`, `StackReq::or_`
`PROXSUITE_CHECK_SIZE`	an explicit `if (...) return false;`
`DENSE_LDLT_FP_PRAGMA`, `LDLT_FN_IMPL3`, `LDLT_LOAD_STORE`, `LDLT_ARITHMETIC_IMPL`	internal; `#undef`'d after use

Renamed:

Was	Is now
`LDLT_TEMP_VEC`, `LDLT_TEMP_VEC_UNINIT`	`PROXSUITE_LDLT_TEMP_VEC`, `PROXSUITE_LDLT_TEMP_VEC_UNINIT`
`LDLT_TEMP_MAT`, `LDLT_TEMP_MAT_UNINIT`	`PROXSUITE_LDLT_TEMP_MAT`, `PROXSUITE_LDLT_TEMP_MAT_UNINIT`
`LDLT_ID`, `LDLT_CONCAT`, `LDLT_CONCAT_IMPL`	`PROXSUITE_LDLT_ID`, `PROXSUITE_LDLT_CONCAT`, `PROXSUITE_LDLT_CONCAT_IMPL`
`__LDLT_TEMP_VEC_IMPL`, `__LDLT_TEMP_MAT_IMPL`	`PROXSUITE_LDLT_TEMP_VEC_IMPL`, `PROXSUITE_LDLT_TEMP_MAT_IMPL`
`LDLT_EXPLICIT_TPL_DECL`, `LDLT_EXPLICIT_TPL_DEF`	`PROXSUITE_EXPLICIT_TPL_DECL`, `PROXSUITE_EXPLICIT_TPL_DEF`

`PROXSUITE_EXPLICIT_TPL_DECL` / `_DEF` support 1 to 3 parameters rather than an arbitrary number. Every call site uses one,

two or three.

5.8 Serialized archives

`Results<T>::active_constraints` and `dense::Workspace<T>::alphas` were written by hand-rolled `save` / `load` overloads on `veg::Vec`. They now go through cereal's `std::vector` support.

Binary archives are unaffected on 64-bit platforms. The old code wrote the element count as an `isize` (`std::ptrdiff_t`), cereal writes it as a `std::uint64_t`; both are eight little-endian bytes for a non-negative count, and the elements follow unchanged. On a 32-bit platform `std::ptrdiff_t` is four bytes, so binary archives written there are not compatible.

JSON and XML archives are not compatible. The old code emitted a named `len` field plus auto-named values, producing a JSON object. Cereal's vector support emits a JSON array:

```
        // before
        "active_constraints": { "len": 2, "value0": true, "value1": false }

        // after
        "active_constraints": [ true, false ]
```

The format carries no version tag, so nothing detects the mismatch. This reaches Python: `Results.__getstate__` / `__setstate__` go through `saveToString` / `loadFromString`, which use the JSON archive. Pickles of `Results`, and of `QP` which holds one, produced by an older ProxSuite will not load. Regenerate them, or read them with the older version and convert.

---

6. Maintainability

The change replaced about 10 300 lines of in-house infrastructure with 467 lines plus the standard library. Effects:

- **Standard types.** `std::vector`, `std::tuple`, `enable_if_t`, structured bindings and `if constexpr` are documented and supported by IDEs and static analysis. The `veg` equivalents were not.
- **Diagnostics.** A wrong type in a `std::enable_if_t` overload produces a candidate list. The same error inside a `VEG_TEMPLATE` produced a macro expansion trace.
- **Tooling.** clang-tidy, sanitizers, IntelliSense and Doxygen handle ordinary templates better than preprocessor output. The `#pragma once` and macro-prefixing commits exist because those problems only became visible once the noise was gone.
- **Dependency graph.** `veg` was included by every header, so any change to it triggered a full rebuild. `dynstack.hpp` is 359 lines with one responsibility.
- **Compile time.** ProxSuite's own share of a translation unit drops 20% (§7.1).
- **One defect found and fixed.** §3.9, in the only place where `veg` carried a hand-rolled serialization path.

Costs:

- `resize` value-initializes where `resize_for_overwrite` did not. Setup paths only; measured in §7.
- The ABI changed (`MatRef` and friends, `Ldlt` storage).
- `std::vector` copies implicitly where `veg::Vec` required an explicit copy (§3.5).

---

7. Performance

Measured on g++ 13.3, `-std=c++17`, Eigen 3.4, one pinned core. "old" is the commit before the port, "new" is `HEAD`. Both header trees compiled from identical sources.

7.1 Compile time

One translation unit including `proxqp/dense/wrapper.hpp` and `proxqp/sparse/wrapper.hpp`, median of 7 interleaved runs:

	old (veg)	new	delta
`-O0`	2.861 s	2.554 s	-10.7%
`-O2`	3.063 s	2.654 s	-13.4%
`-O2`, minus the Eigen-only baseline (1.228 s)	1.889 s	1.512 s	**-20.0%**

ProxSuite's own share of the compile drops by a fifth. The preprocessed line count barely moves (247 292 -> 242 120, -2%) because Eigen dominates it; the saving is in template instantiation and preprocessor expansion, not in lines fed to the parser.

7.2 Solver runtime

Catch2 `BENCHMARK_ADVANCED`, `-O3 -DNDEBUG`, 3 independent runs of 40 samples, median across runs. Each sample constructs a fresh `QP` so that `init` is measured cold.

Benchmark	old (veg)	new	delta
dense init (n=100)	398 us	402 us	+0.8%
dense init+solve (n=100)	1.83 ms	1.83 ms	-0.4%
dense init+solve (n=300)	14.04 ms	14.68 ms	+4.6%
sparse init (n=200)	849 us	891 us	+5.0%
sparse init+solve (n=200)	18.10 ms	18.47 ms	+2.0%

There is no measurable change in solver runtime, in either direction. Run-to-run spread is 1-8% depending on the case, which covers every delta in the table. These numbers were taken on a shared 2-core container; treat them as evidence of no regression, not as a measurement of one.

No change was expected. The allocator swap is a no-op (§3.5), and niebloids to function templates is a source-level change with identical codegen. The one mechanism that could cost anything is the `resize_for_overwrite` to `resize` zero-fill on `init`, which scales with workspace size and does not touch solve iterations. If it is ever worth chasing, run these benchmarks on real hardware first; the fix would be C++23's `resize_and_overwrite`, not a custom allocator.

---

8. Future maintenance

8.1 Dependencies and macros

1. **No new vendored utility layers.** Prefer the standard library, then Eigen, then a small self-contained header in `proxsuite/helpers/`. `dynstack.hpp` sets the pattern: one responsibility, no dependencies beyond `<new>` and `<cassert>`, documented at the top.
2. **Every macro in an installed header carries the `PROXSUITE_` prefix, or is `#undef`'d before the end of the header.** A CI grep over the installed headers for `^#define` without the prefix would enforce this; the packaging tests are the natural place for it.
3. **Macros must not hide control flow.** `PROXSUITE_CHECK_SIZE` expanded to a bare `return false`, so a call site that read like a statement could exit the function. If a macro cannot be read as an expression or a declaration, write the code out.

8.2 C++20 and later

4. **Replace the six `enable_if` constructors with `static_assert`.** This removes the last `std::enable_if_t` in the codebase and needs no macro and no C++20. The signature collapses to one template parameter:

```
// now
template<typename Mat,
         typename = std::enable_if_t<concepts::eigen_view<Mat, T> &&
                                     eigen::GetLayout<unref<Mat>>::value == L>>
PROXSUITE_INLINE MatrixView(FromEigen /*tag*/, Mat const& mat) noexcept
  : data(mat.data()), rows(mat.rows()), cols(mat.cols())
  , outer_stride(mat.outerStride())
{
}

// proposed
template<typename Mat>
PROXSUITE_INLINE MatrixView(FromEigen /*tag*/, Mat const& mat) noexcept
  : data(mat.data()), rows(mat.rows()), cols(mat.cols())
  , outer_stride(mat.outerStride())
{
  static_assert(concepts::eigen_view<Mat, T>,
                "Mat must be an Eigen matrix whose data() yields T const*");
  static_assert(eigen::GetLayout<unref<Mat>>::value == L,
                "Mat's storage order must match the view's layout");
}
```

Verified by patching all six sites: the dense and sparse solvers and the benchmark compile clean, invalid arguments are still rejected, and the diagnostic improves from 32 lines of substitution failures to 7 lines whose first error is the assertion text.

The one behavioural change is that `std::is_constructible` no longer instantiates the body, so it reports `true` for arguments the constructor would reject, including `MatrixView<T, colmajor>` from `FromEigen, int`. Nothing in the repository or the bindings queries these traits, and the `FromEigen` tag keeps the constructors out of implicit-conversion paths, so the loosening is not reachable in practice. Confirm before applying.

Note for later: the `concepts` predicates already work as C++20 constraints unchanged, since a `constexpr bool` variable template is a valid constraint-expression. A `PROXSUITE_REQUIRES` macro that expands to `requires(...)` on C++20 and to nothing otherwise is possible — guard on `__cpp_concepts`, not `__cplusplus`, which MSVC reports as `199711L` without `/Zc:__cplusplus`. It is not worth adding: a vanishing macro is only correct where the constraint is not doing SFINAE, and keeping SFINAE on C++17 means a second macro and the condition written twice. Writing it once would require the macro to consume the closing `>` of the template parameter list, which is the `VEG_TEMPLATE` design this change removed.

5. **`Slice` / `SliceMut` are `std::span` in all but name**, and `DynStackMut` is a monotonic buffer resource. Not worth changing now, but if the baseline moves, deleting `slice.hpp` is a small change rather than a rewrite.

6. **Revisit `resize` vs. uninitialized resize only if profiling justifies it.** `resize_for_overwrite` is `resize_and_overwrite` in C++23; until then the zero-fill does not justify a custom allocator.

8.3 General

7. **Assertions stay in `assert`.** Sanitizers and `NDEBUG` already handle it. Do not reintroduce an assertion framework.
8. **Keep the ABI notes in the changelog current.** `MatRef` and friends changed layout here. Downstream packagers need that signal, and it is cheap to write at the time and expensive to reconstruct later.
9. **Delete the stray `row_matrix` file at the repository root**, a serialization test artifact committed by accident, and add the pattern to `.gitignore`.